

How we built a depth-95 phase oracle for the Classiq logo

One sandwich, two small codes, and a search over the codes instead of the circuit

PoJen Wang
pwang529@gmail.com

Abstract

The Classiq challenge asks for a quantum circuit that puts a minus sign on the 1097 black pixels of a 64×64 logo, using `u3` and `cx` gates on at most 18 qubits, scored by depth. The organisers' reference circuit has depth 5329. Ours has depth 95 and 272 `cx`. This note explains the method in four steps. First, the whole logo is handled by one *compute, flip the sign, uncompute* sandwich, whose compute step splits into a code read from the column bits and a code read from the row bits; the two codes sit on different qubits, so they run at the same time. Second, the codes are small: a column says how far from its centre line it still reaches, a row says how far it sits from its centre line, and the middle of the sandwich compares the two. Third, we list every code word each column and each row may use, which makes the two codes independent, so any legal column code works with any legal row code. Fourth, a simulated annealer searches for short codes millions of times per second, steered by a cheap depth estimate that we trust for direction but never for the final choice. A last round of exact rescheduling and SAT re-synthesis takes the circuit from 107 to 95.

Contents

1	The method on one page	1
2	The problem	3
3	One sandwich for the whole logo	4
4	Row and column codes	5
5	Contracts and pairing	9
6	Searching the codes instead of the circuit	11
7	From 107 to 95: polishing a finished circuit	14
8	What did not work, and what we would keep	16
	References	18

1 The method on one page

Our final circuit has depth 95 with 272 `cx`, against 5329 and 3502 for the organisers' reference circuit [1]. Most of that gap comes from four ideas, which [figure 1](#) lays out with the depth each one bought.

1. **One sandwich for the whole logo** ([section 3](#)). Compute a little information about the pixel, flip the sign once, then undo the computation. The compute step is split into a part that reads only the column x and a part that reads only the row y . They sit on different qubits, so they run at the same time.

2. **Row and column codes** (section 4). The column part writes a 4-bit *radius*, the row part a 4-bit *distance*, and a pixel is black when the distance fits inside the radius. A few flag bits handle the square and the bar.
3. **Contracts and pairing** (section 5). We list every code word each column and each row may produce without breaking the picture. With those lists, any legal column code works with any legal row code, so the two halves are searched separately.
4. **Searching the codes instead of the circuit** (section 6). A simulated annealer edits code circuits millions of times per second, guided by a cheap depth estimate. The estimate decides where to look; the real compiler decides what to keep.

A final round of exact rescheduling and SAT-based rewriting (section 7) took the circuit from 107 to 95 layers without changing the codes.

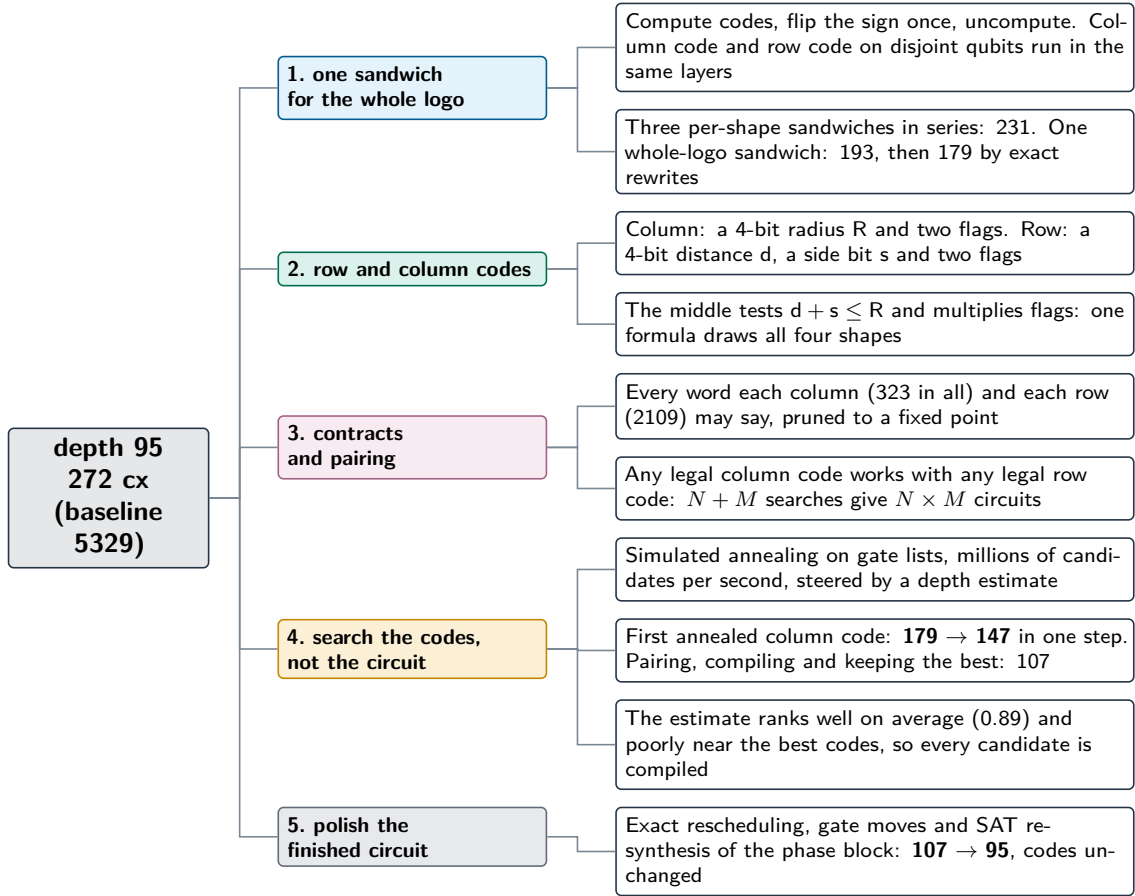


Figure 1: The method as a map. Each branch is one idea of the method, in the order the sections present them, with the depth it bought. Ideas 1 to 4 design the circuit; idea 5 packs a finished circuit more tightly. The baseline is the organisers’ reference circuit.

Who this is for. We assume you know what a qubit, a cx gate and a circuit diagram are. We do not assume any background in compilers or logic synthesis; every such idea is explained where it first appears.

Companion notebook. The notebook `classiq-challenge-depth-95-explainer.ipynb` ships with this report, together with one helper script and a `data/` folder that holds every circuit and search log it reads. It recomputes every plot in this note in under a minute on a laptop, and runs a short annealing search live.

2 The problem

2.1 A phase oracle for a picture

Put the column $x \in \{0, \dots, 63\}$ of a pixel on six qubits and the row y on six more. A *phase oracle* for the logo, the kind of circuit Grover’s search algorithm queries [2], leaves every basis state as it is except for a sign:

$$U_f |x\rangle|y\rangle|0\rangle^{\otimes 6} = (-1)^{f(x,y)} |x\rangle|y\rangle|0\rangle^{\otimes 6}, \quad (1)$$

where $f(x, y) = 1$ on a black pixel. The six extra qubits are *helpers*. The circuit may use them as scratch space, but they must end in $|0\rangle$ again, so that the oracle behaves the same on any superposition of pixels. That gives 18 qubits in all: x on q_0 – q_5 , y on q_6 – q_{11} , and helpers on q_{12} – q_{17} .

The rules are simple. The circuit may contain **u3** gates (any one-qubit gate) and **cx** gates between any two qubits. It is scored by its **depth**, the number of layers of gates, with the **cx** count as a tie breaker. Two gates can share a layer when they act on different qubits.

The logo (figure 2) is the union of four shapes that the challenge defines exactly: a square ($2 \leq x \leq 26$, $29 \leq y \leq 53$), a bar ($26 \leq x \leq 49$, $39 \leq y \leq 43$), a small disc D_1 with $(x - 55)^2 + (y - 41)^2 \leq 42$, and a big disc D_2 with $(x - 40)^2 + (y - 19)^2 \leq 72$. Together they light 1097 of the 4096 pixels.

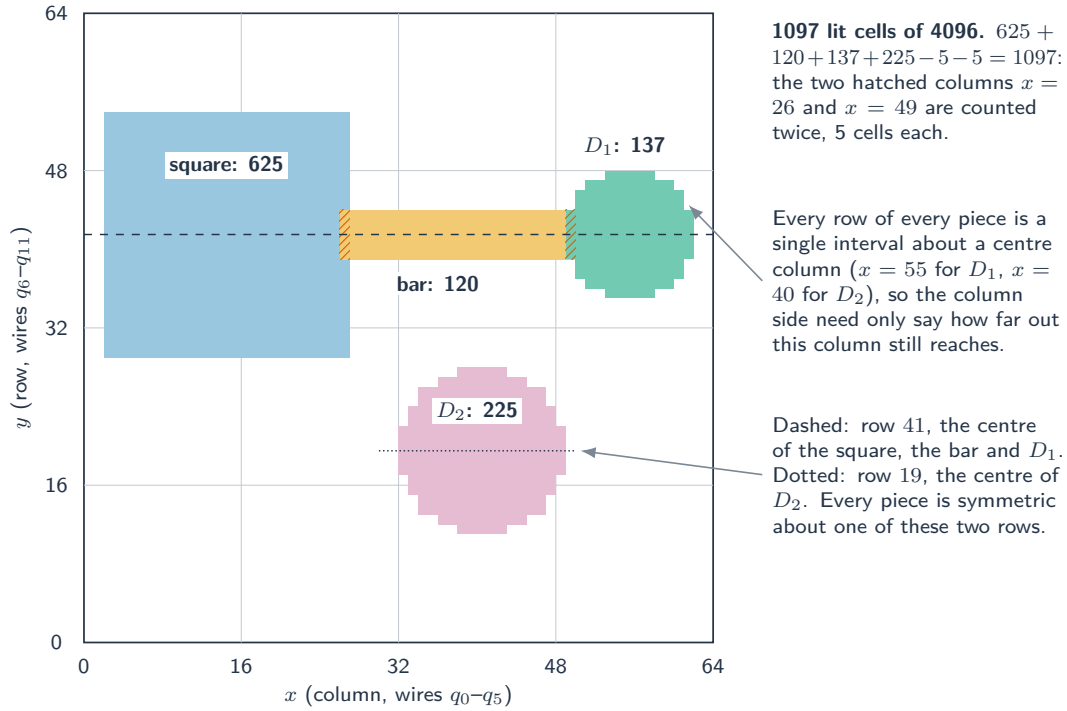


Figure 2: The target: four overlapping shapes on a 64×64 grid. Each shape is symmetric about one of two rows, row 41 (dashed) or row 19 (dotted), and in every column each shape covers a single run of rows. The codes of section 4 are built on these two facts.

2.2 Depth is about packing

Depth counts layers, and one layer can hold many gates as long as no two of them touch the same qubit. So two pieces of work on *disjoint* qubits cost the longer of the two, while two pieces on the *same* qubits cost the sum:

piece A on q_0, q_1, q_2 : 3 layers	piece B on q_3, q_4, q_5 : 2 layers	A then B: 3 layers
piece A on q_0, q_1, q_2 : 3 layers	piece C on q_0, q_1, q_2 : 3 layers	A then C: 6 layers

This one rule shaped the whole design. The organisers’ reference circuit covers the logo with 18 rectangles and marks them one at a time. Every rectangle needs all twelve coordinate qubits, so the rectangles cannot overlap in time and their depths add up to 5329. We wanted a circuit whose expensive parts live on *different* qubits, so that a scheduler can stack them in the same layers.

3 One sandwich for the whole logo

3.1 Compute, flip the sign, uncompute

Almost every oracle that uses helper qubits has the same three-part shape, often called *compute–uncompute* [3]:

- E compute some useful bits from x and y onto the helpers (the *encoder*);
- Φ flip the sign, depending on those bits (the *middle*);
- $E^\dagger$ run the encoder backwards, which returns the helpers to $|0\rangle$.

The middle only multiplies each basis state by ± 1 , so running E backwards afterwards undoes everything except the sign. We call this shape a *sandwich*. A bonus of the exact undo is that the encoder’s multi-controlled gates may use cheaper forms that are right only up to a phase, since the undo cancels that phase [4].

Our first circuits used one sandwich per shape and ran them one after another. Each sandwich needed all twelve coordinate qubits, so, by the packing rule of [section 2](#), their depths added up. That design stopped at 231 layers ([figure 3a](#)).

The design that won uses **one sandwich for the whole logo** and splits its encoder in two ([figure 3b](#)):

- a *column code* E_{col} , which reads only the six x qubits and writes on four helpers;
- a *row code* E_{row} , which reads only the six y qubits and writes on the other two helpers.

The two codes share no qubit, so they run in the same layers, and the encoder costs the longer of the two instead of their sum. An early whole-logo circuit started at 260 layers. We then let an LLM-driven program-evolution loop, ShinkaEvolve [5] (in the spirit of FunSearch [6]), propose rewrites of the construction, each scored on the full compiled circuit. Together with a few rewrites by hand, this took the circuit to 202 layers, already shorter than 231 although its codes were crude.

Two pricing rules follow from this shape, and the rest of the method leans on both:

Two pricing rules of the sandwich

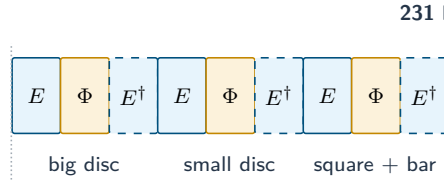
1. Every layer of a code is paid **twice**, once on the way in and once on the way out. A layer of the middle is paid once.
2. The two codes run in parallel, so the circuit is roughly as deep as its **longer side**. Shortening the shorter side buys nothing.

3.2 Where the outputs live

There are no spare qubits: twelve hold the coordinates and only six are helpers. So each code writes its outputs partly on top of its own input qubits, and only the exact undo at the end puts x and y back. [Figure 4](#) shows where every output bit sits in the final circuit. The column side

(a) one sandwich per shape: 231 layers

all 18 wires: every block needs the same twelve coordinate wires



231 layers

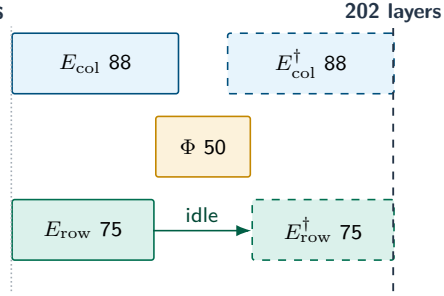
Serial by construction. Each shape block writes its code on the coordinate wires, uses it, and puts them back. The next block needs the same wires, so it cannot start early. Three sandwiches, paid one after another.

(b) one sandwich for the whole logo: 202 layers

column side, 10 wires

the middle, reading both sides

row side, 8 wires



202 layers

Concurrent by construction. One column code and one row code serve all four shapes at once, and they touch disjoint wires, so they occupy the same layers. $202 = 88 + 50 + 88 - 24$: the compiler even slides the middle into the tail of the code. This is an early whole-logo circuit; later codes are much shorter.

time, one layer = 0.25 mm; both panels to the same scale

Figure 3: Serial sandwiches against one shared sandwich, drawn to one time scale. (a) Three per-shape sandwiches each need every coordinate qubit, so they run in series. (b) One sandwich for the whole logo: the column code and the row code sit on disjoint qubits and run side by side, and the middle reads both. Even this early version, with long codes, is shorter.

owns ten qubits (x plus q_{14} – q_{17}) and the row side owns eight (y plus q_{12} , q_{13}). A single gate, $\text{ccx}(q_0, \overline{q_{11}} \rightarrow q_3)$, reads both sides; it saved six cx late in the campaign.

3.3 The finished circuit, layer by layer

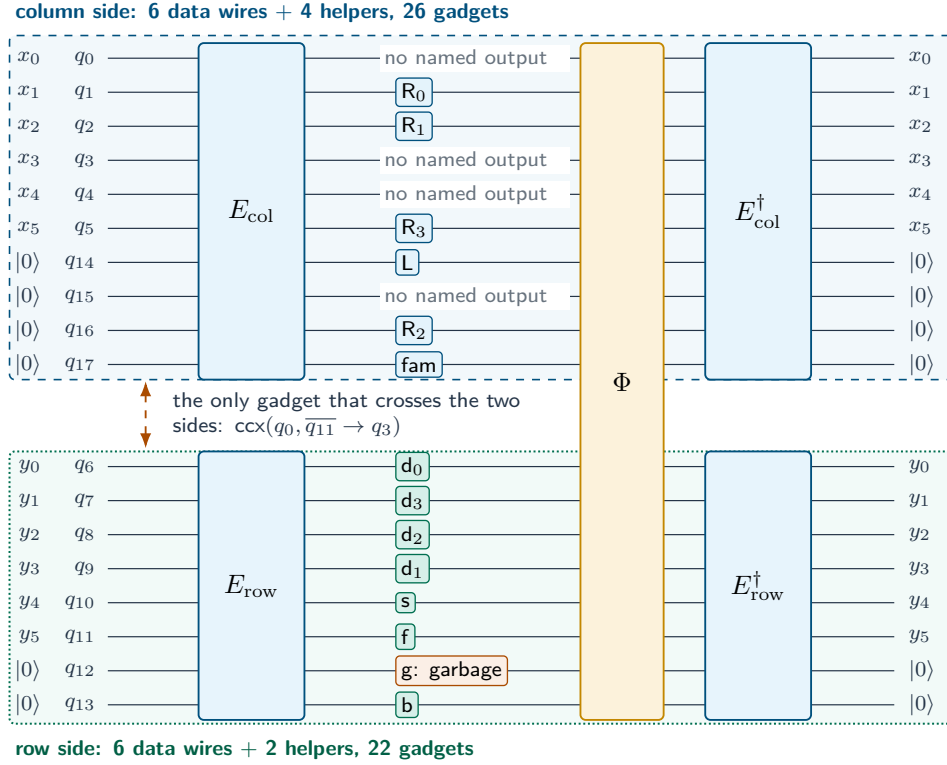
Figure 5 draws the final 95-layer circuit as a sandwich, and figure 6 shows every one of its 527 gates in the layer where it runs. The sandwich is easy to see. Layers 1–43 hold 131 of the 272 cx, mostly the two codes, with the column side (top ten rows of the plot) and the row side (bottom eight) in the same layers. Layers 44–52 are mostly the middle and hold only 13 cx. Layers 53–95 hold 128 cx, and 127 of them also appear, with the same control and target, in layers 1–43: they are the codes run backwards.

Figure 7 counts the same gates per qubit. The column helpers q_{14} – q_{17} and the top row bit q_{11} carry the most work, and q_{16} is busy in 81 of the 95 layers, while q_0 and q_7 are busy in only 18. Depth is set by a few crowded wires, which is why the search in section 6 targets the column code and the time its helpers stay busy.

4 Row and column codes

4.1 What should the codes compute?

The sandwich only helps if both codes are short, so the codes should carry as little information as the middle needs. Figure 2 suggests what that is. Look at the logo one column at a time. In every column, the black pixels of each shape form one unbroken run of rows, centred on row 41 or on row 19. So a column can be summarised by *how far from the centre row it still reaches*, a row by *how far it sits from the centre row*, and a pixel is black when the second number is within the first.



The outputs sit on the data. Three column outputs land on x wires and six row outputs land on y wires; nothing is copied to a spare wire, because there are no spare wires. Only the exact adjoint puts x and y back.

Four wires carry nothing named, and q_{12} carries garbage that no later gate reads. It still has to be held to the end and cleaned, which is why it is the wire both sides fight over.

Figure 4: Which qubit holds which output between a code and its undo, in the final circuit. Column outputs (blue tags) and row outputs (green tags) are explained in [section 4](#). The column side uses ten qubits and the row side eight; one gate crosses between them.

	bits	meaning
column code	$R = R_0 R_1 R_2 R_3$	a 4-bit <i>radius</i> : how far from the centre row this column belongs to a disc
	fam	“this is a bar column”
	L	“this is a disc column”
row code	$d = d_0 d_1 d_2 d_3$	a 4-bit <i>distance</i> from this row to its centre row (41 or 19)
	s	1 if the row is above its centre row
	f	“this row crosses the square” ($29 \leq y \leq 53$)
	b	“this row crosses the bar” ($39 \leq y \leq 43$)

One distance for two centre rows. The row code has to measure distances to two different centre rows with the same four bits. It does this with a subtraction ([figure 8](#)). It computes $\text{centre} - y$ in four-bit binary and keeps the sign of the result as s . On or below the centre row the result is the distance. Above it the result is negative, and its bits read one less than the distance. Instead of spending gates to fix this, the middle tests $d + s \leq R$, which is the true distance on both sides.

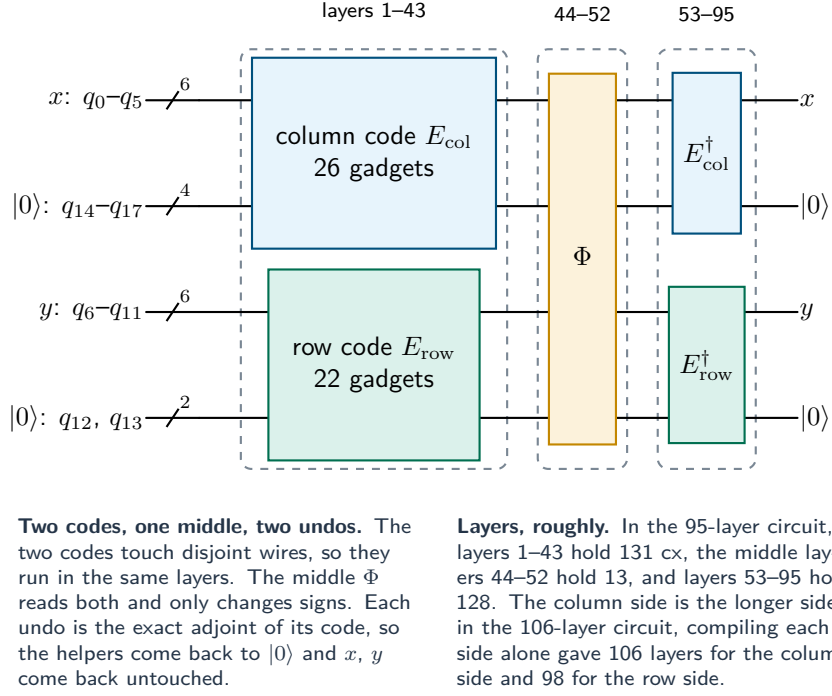


Figure 5: The whole oracle is one sandwich. Two codes side by side, one short middle Φ that reads both, and the exact undo of each code. The layer ranges are those of the final 95-layer circuit.

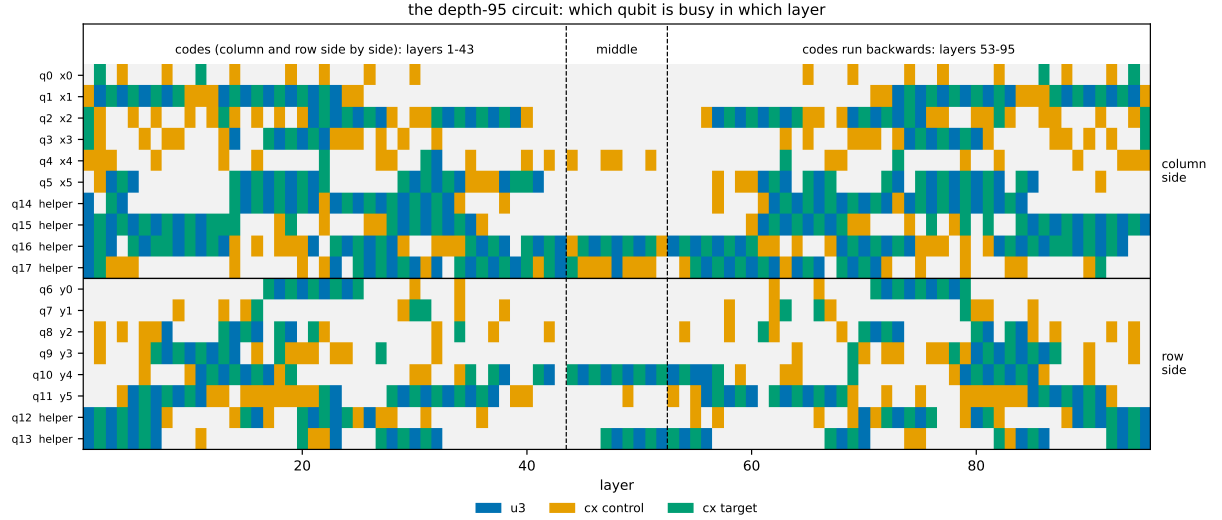


Figure 6: The final circuit, qubit by qubit and layer by layer. A coloured cell is a gate on that qubit in that layer. The left and right blocks are near mirror images (the codes and their undo); the middle block is short. The helper q_{16} is busy in all nine middle layers, which is what stops the circuit at 95 (section 7).

4.2 The middle: one formula for four shapes

The middle reads the thirteen code bits and flips the sign when

$$\Phi = \underbrace{\text{f a m b f}}_{\text{bar}} \oplus \underbrace{(\text{f} \oplus \text{f a m}) \text{L X C}}_{\text{discs}} \oplus \underbrace{\text{L R}_3}_{\text{correction}} \oplus \underbrace{\text{f R}_3}_{\text{square}} \quad (2)$$

is 1, where $X = d_3 \oplus \overline{R_3}$ and $C = 1[(d \bmod 8) + s \leq (R \bmod 8)] \oplus R_3$. Products are ANDs of bits, and $\oplus$ is addition mod 2.

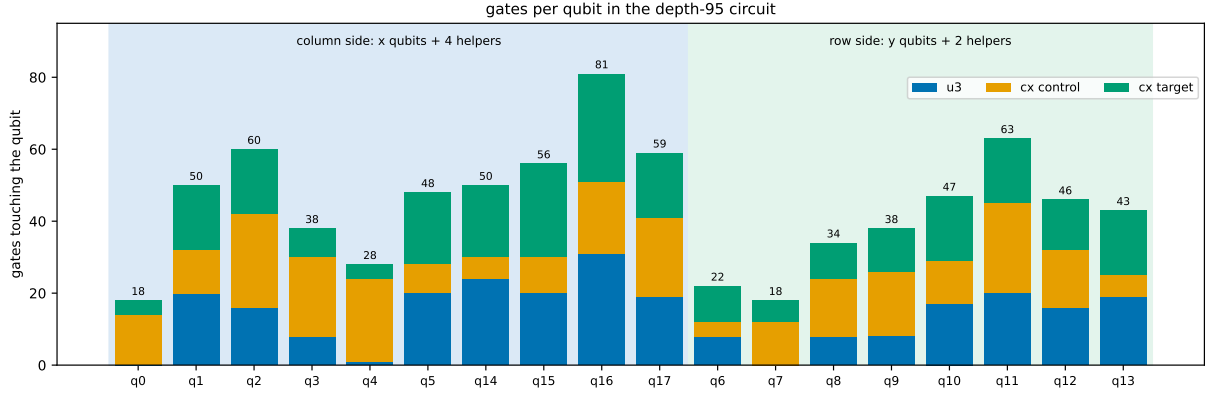
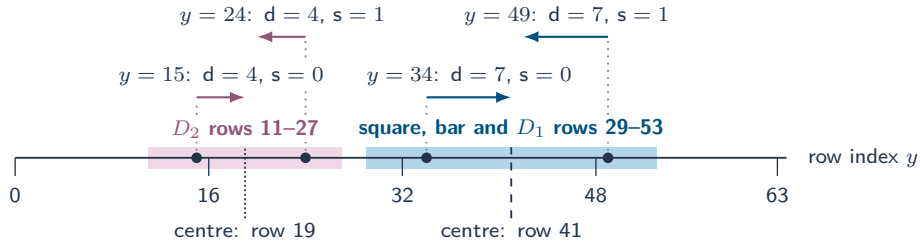


Figure 7: Gates per qubit in the final circuit, split into u3, cx control and cx target. The column side (blue background) carries more work than the row side (green background), and the helper q_{16} the most of all.

(a) the row y , its two bands, and the distance stored



(b) why one bit flip cannot serve both bands

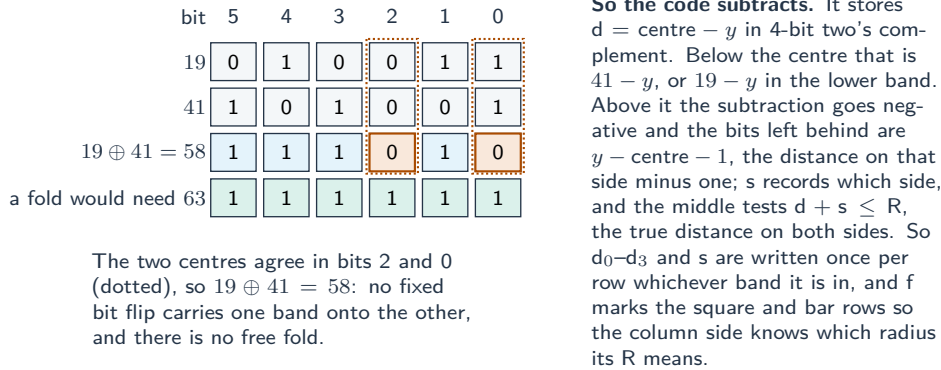


Figure 8: One 4-bit distance for both centre rows. (a) Rows of the big disc measure from row 19; rows of the square, bar and small disc from row 41. (b) A simpler trick, flipping bits of y , would need the two centre rows to differ in every bit, and 19 and 41 do not. So the code subtracts, keeps the sign bit s , and the middle compares $d + s$ with the radius.

Figure 9 draws each term as a picture. Three of them need no arithmetic. The bar is a column flag times two row flags, and the square is a row flag times the top radius bit. The correction term covers the five columns $x = 38, \dots, 42$, where the big disc's half-height is 8 and so needs the fourth radius bit. The disc term holds the one real comparison: on a disc column with radius below 8 it is exactly $1[d + s \leq R]$. That comparison is why the middle contains a small adder-like circuit (a *ripple-carry comparator* [7]), and it is the most expensive part of the middle.

Why XOR rather than OR? A pixel covered by two terms cancels, and the formula is written so that the cancellations land exactly on the white pixels. In return, a sign flip by an XOR of

products splits into one sign flip per product, because $(-1)^{a \oplus b} = (-1)^a (-1)^b$, so the middle can apply the terms one at a time. This is the standard way to turn an XOR of products into a phase oracle [8].

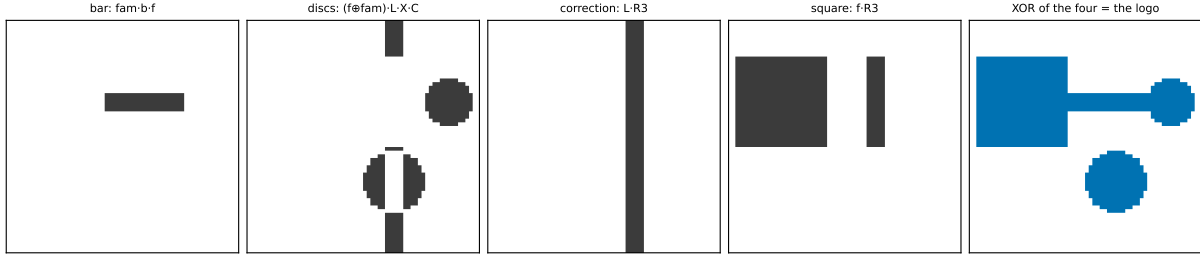


Figure 9: The four terms of the middle’s formula (2), evaluated on the real codes of one of our circuits, and their XOR. Each black region is where that term equals 1. Overlaps outside the logo cancel, and the XOR (right) is the logo with no wrong pixel.

Worked example: three pixels through the codes

The values below are the real outputs of the codes in our depth-138 circuit, whose column and row codes were found by two separate searches.

pixel (x, y)	R	fam	L	d	s	f	colour
(55, 44), small disc	6	0	1	2	1	1	$d + s = 3 \leq 6$: black
(55, 48), above it	6	0	1	9	0	1	$d + s = 9 > 6$: white
(10, 30), square	13	0	0	11	0	1	$f R_3 = 1$: black

At $x = 55$ the small disc reaches 6 rows from row 41, and $R = 6$ says so. Row 44 is 3 above row 41: the code stores $d = 2$, $s = 1$, and $2 + 1 \leq 6$. Row 48 is 7 away. The code stores 9, which is not the true distance but is just as good, since anything above 6 gives the right answer here. In the square column $x = 10$ the radius is 13, but the middle only reads its top bit $R_3 = 1$, and $f R_3$ marks the pixel black.

4.3 The codes themselves

Figure 10 plots the two codes of the same circuit next to the numbers they stand for. Where a disc needs the number, the code is exact: the radius matches the disc’s true half-height in every disc column, and the distance matches the true distance in every row a disc covers. Elsewhere the code wanders. Under the square the radius takes any value from 10 to 15, because only R_3 is read. Far from every disc the distance only has to be large enough. The search of section 6 found these codes, and it used that slack to make them shorter. Section 5 turns the slack into a tool.

5 Contracts and pairing

5.1 Writing down the freedom

The square columns showed that a code has choices: wherever the middle does not read a bit, any value works. We made this exact. For every column x we list the set $\mathcal{A}_x$ of six-bit words the column code may write there, and for every row y the set $\mathcal{A}_y$ of words the row code may write. (The row code has one spare output bit g that the middle never reads, so row words have eight bits.) A word is allowed if it still draws the right picture. Logic synthesis calls such free outputs *don’t-cares*, and exact synthesis tools exploit them in the same way [9].

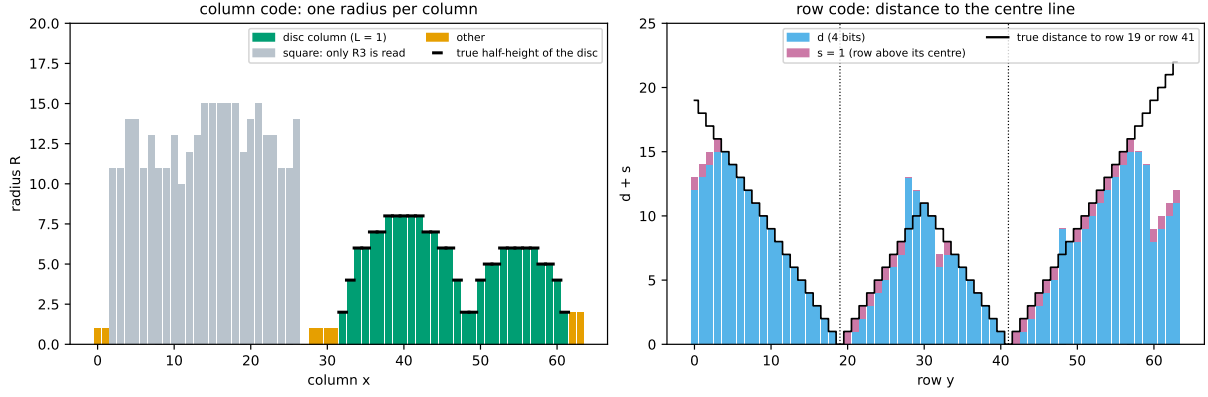


Figure 10: The two codes of our depth-138 circuit. Left: the radius R written for each column; black ticks mark the disc’s true half-height, which R matches on every disc column. Right: $d + s$ written for each row (the purple cap is s); the black line is the true distance to row 19 or row 41. The codes agree with the geometry where a disc reads them and are free to differ elsewhere.

There is a catch. A column word is only right or wrong *together with* a row word, so the two lists depend on each other. We settle that by pruning until nothing changes:

```

start   every column may write any 6-bit word, every row any 8-bit word
repeat  drop a column word if it draws a wrong pixel against SOME
        allowed word of SOME row
        drop a row word if it draws a wrong pixel against SOME
        allowed word of SOME column
until   nothing is dropped

```

What survives is the **contract** (figure 11, left). For the middle of equation (2) it allows 323 column words, between 1 and 9 per column, and 2109 row words, between 4 and 60 per row. The column side splits into four groups:

columns	allowed words	what is fixed
0–1, 62–63 (empty edges)	8	$R_3 = \text{fam} = L = 0$
2–26 (square)	8	$R_3 = 1, \text{fam} = L = 0$
27–31, 49, 61 (bar only, disc edges)	9	only $R_3 = 0$
32–48, 50–60 (discs)	1	all six bits

This reads like a summary of the picture. Under the discs the radius must be exact. Under the square only R_3 matters. Rows far from every shape may use up to 60 different words.

5.2 Why the contract makes pairing free

Because the pruning ran until nothing changed, every surviving column word is right against *every* surviving row word. So:

Pairing

Any column code that writes allowed words on all 64 columns works with any row code that writes allowed words on all 64 rows. With N column codes and M row codes, $N + M$ independent searches give $N \times M$ correct circuits.

This splits one hard joint search into two smaller ones that can run on different machines and never talk to each other (figure 11, right). Checking a candidate code also becomes cheap. A column code is correct if it writes an allowed word on each of the 64 columns, which is 64 table lookups instead of 4096 pixels.

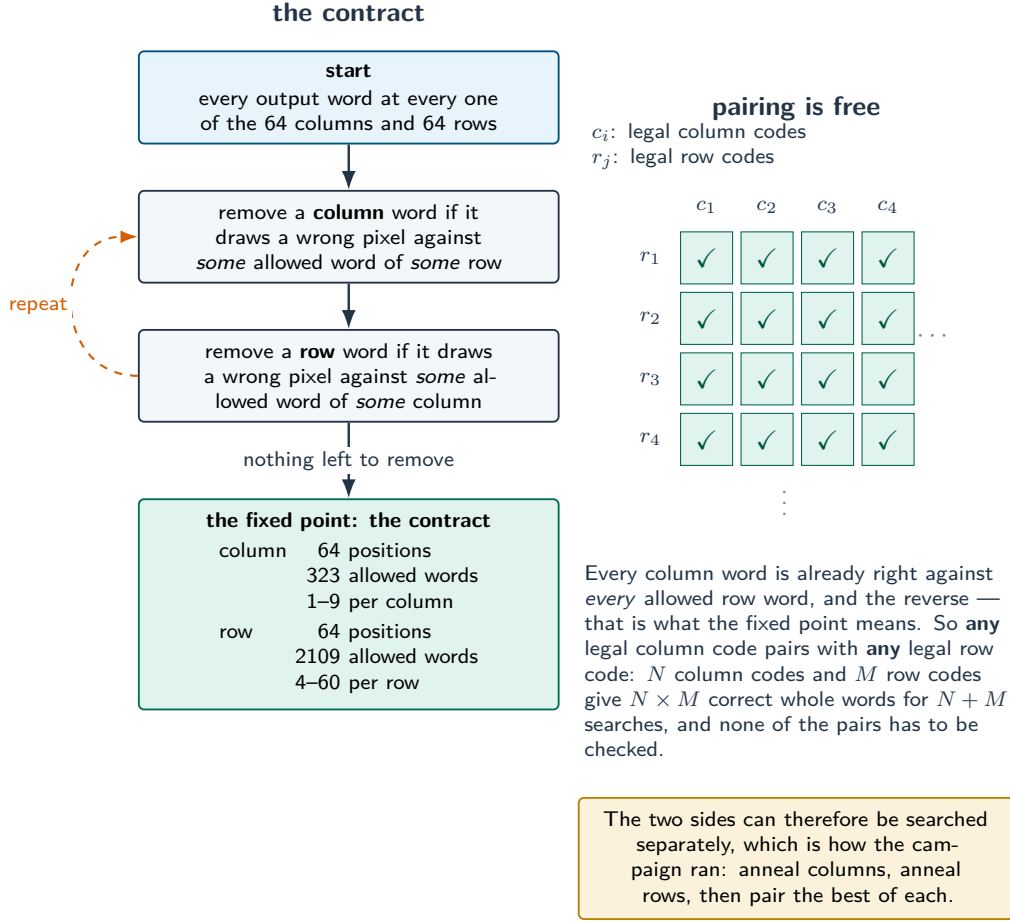


Figure 11: The contract and why it makes pairing free. Left: allowed words are pruned until nothing changes, and what remains is the contract. Right: because each column word is right against every allowed row word, any legal column code pairs with any legal row code.

5.3 Pairing in practice

Figure 12 shows one real pairing run from the campaign: 6 annealed column codes against 4 annealed row codes, all 24 combinations built and compiled. Every pair draws the logo with no wrong pixel. Changing the column code moves the depth by up to 2 layers. Three of the four row codes give identical depths and the fourth adds one layer everywhere. This is the second pricing rule at work: the column side is the longer side, so a better row code buys nothing until the column code improves. From here on, the campaign scored each side alone by compiling a *half circuit*: one code, the middle, and that code’s undo.

6 Searching the codes instead of the circuit

Hand-designed codes took the whole-logo sandwich to 179 layers. Below that, the codes were found by a computer. This section describes the search, the cheap depth estimate that steers it, and how much that estimate can be trusted.

6.1 What is being searched

A column code is a small reversible circuit on ten qubits: the six x qubits and four helpers that start at 0. It is built from x , cx , ccx and $c3x$ gates (a NOT with zero to three controls, any of which may be negated), plus a choice of which qubit holds each of the six outputs

24 pairs, all correct: compiled depth of each pair

row 1	133	132	133	134	133	133
row 2	133	132	133	134	133	133
row 3	133	132	133	134	133	133
row 4	134	133	134	135	134	134
	col 1	col 2	col 3	col 4	col 5	col 6

Figure 12: Twenty-four paired circuits, all correct. Each cell is the compiled depth of one column code (columns) paired with one row code (rows) from a single pairing run. Most of the variation follows the column code.

$R_0, \dots, R_3, \text{fam}, L$ and whether it is stored inverted. The code is correct if, on every column, it writes a word the contract allows.

We search this space with *simulated annealing* [10], a random walk that proposes a small change, always accepts it if the result is better, and sometimes accepts it if it is worse. The chance of accepting a worse change shrinks as a “temperature” is lowered, which lets the walk climb out of dead ends early on and settle down later. Our program, `colsa.c`, is about 300 lines of C. Its moves change a control, flip a control’s polarity, change a target or a gate type, insert, delete, swap or move a gate, or move an output to another qubit. The idea is close to *stochastic superoptimisation* [11], which searches machine code the same way, scoring each candidate on test cases plus a cheap cost estimate; annealing has also been used to synthesise quantum circuits directly [12].

Correctness in a few instructions. Each candidate is checked on all 64 columns at once. Every qubit holds one 64-bit integer whose bit x is that qubit’s value on column x , so a gate is one AND and one XOR over all columns together:

Checking a candidate on all 64 columns at once (from `colsa.c`)

```
for (int k = 0; k < s->n; k++) {                                /* run the gate list */
    const Gate *g = &s->g[k];
    uint64_t m = ~0ULL;                                         /* columns where all controls */
    for (int j = 0; j < ncontrols(g->type); j++)               /* are satisfied ... */
        m &= g->p[j] ? ~v[g->c[j]] : v[g->c[j]];
    v[g->t] ^= m;                                                /* ... get their target flipped */
}
/* then read the six output wires and count columns whose word is not allowed */
```

A depth estimate in a few more. Compiling a candidate into `u3` and `cx` gates and scheduling it takes seconds. (Our compile chain runs a `tket` [13] peephole pass, the Qiskit transpiler [14] at its highest optimisation level, and then our own scheduler, which reorders gates that commute.) The annealer needs an answer in nanoseconds, so it uses an estimate, `est`: the as-soon-as-possible depth of the gate list with fixed prices of 1 layer per `cx`, 7 per `ccx` and 13 per `c3x`, since a multi-controlled gate becomes a block of several layers once compiled [15]. A refined version also charges for how late each output becomes ready. The middle reads some outputs early, and an output it waits for delays the middle and, through the undo, the end of the circuit as well.

The energy. The walk minimises

$$E = 20 \cdot \text{wrong} + 3 \cdot \max(0, \text{est} - \text{cap}) + 0.01 \cdot \text{est} + 0.001 \cdot \text{gates}, \quad (3)$$

where “wrong” counts columns whose word breaks the contract. Each time the walk finds a correct code with $\text{est} \leq \text{cap}$, it prints the code and lowers the cap to $\text{est} - 1$, so it keeps pushing for a shallower code:

The acceptance rule and the cap ratchet (from `colsa.c`, shortened)

```
double T = T0 * pow(T1 / T0, (double)it / iters);           /* cool down slowly */
State t = s; mutate(&t);                                     /* propose a change */
double e2 = energy(&t, cap, &w2, &est2);
if (e2 <= e || runif() < exp((e - e2) / T)) {               /* Metropolis rule */
    s = t; e = e2;
    if (w2 == 0 && est2 <= cap) { print_state(&s, est2, it); cap = est2 - 1; }
}
```

One core checks between two and three million candidates per second this way.

6.2 What the search found

The first large run used 24 chains of 300 million steps each: 12 from random gate lists and 12 seeded with our best hand-made column code (figure 13). Most random chains never reached a correct code, and the three that did got there late. But those three found codes with estimates of 69, 72 and 76. The seeded chains all repaired the hand-made code (which broke the contract on 5 columns) and then stalled between 78 and 96: starting from our own design kept the search close to our own design.

The code with estimate 72 became a circuit of depth 147, thirty-two layers below the hand-made 179 in one step. Annealing the row side the same way and pairing gave 138. From there, rounds of *anneal*, *compile*, *keep the best* on each side, a SAT-scheduled phase block, and the refined estimate brought the circuit to 107.

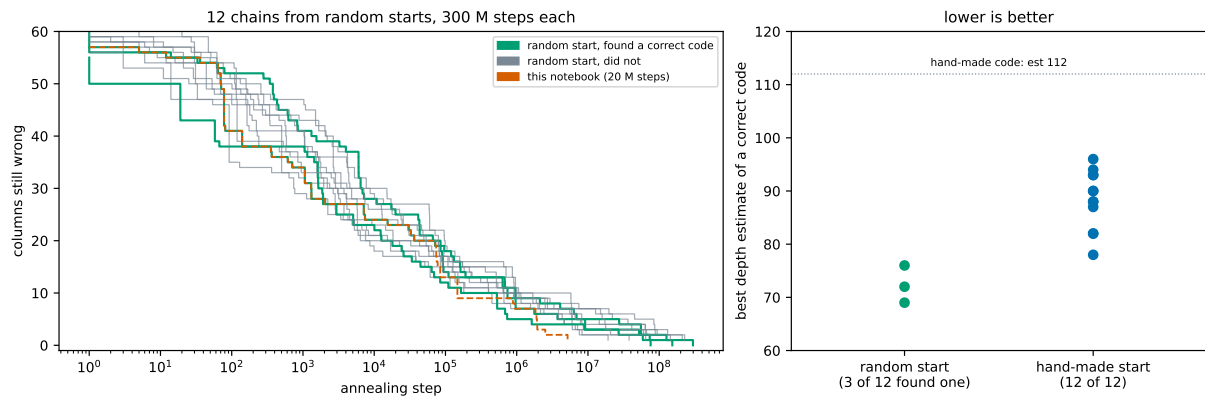


Figure 13: The first annealing run. Left: columns still wrong against annealing step for the 12 random-start chains (log scale). Green chains reached a correct code; the dashed orange trace is a 20-million-step chain run live in the companion notebook. Right: the best estimate of any correct code per chain. Random starts found fewer correct codes but much better ones.

6.3 Is the estimate a good stand-in for depth?

The annealer never sees the real depth, so the method rests on one question: does a lower est mean a shallower circuit? There are three reasons to expect it.

- **The codes are most of the circuit.** In the 106-layer circuit, removing the middle’s phase block left 93 layers. In the 95-layer circuit, everything except that block fits in 86 layers.
- **The prices roughly follow the compiler.** A multi-controlled gate costs several layers once compiled, and the estimate charges it that way.
- **Codes are paid twice.** By the first pricing rule, a layer saved in a code saves two layers of the circuit, so the estimate measures the part of the circuit that counts double.

Figure 14 tests it. On the left, codes from the campaign’s archive of 296,770 column codes are grouped by estimate and compiled. Lower estimate means shallower circuits: the median compiled half circuit climbs from 111 to 131 layers as the estimate goes from 57 to 64. Over 13,830 compiled half circuits, the refined estimate ranks codes in nearly the right order (rank correlation 0.89, against 0.87 for plain as-soon-as-possible depth).

On the right is the catch. Sixty codes with the same estimate compiled to anything from 127 to 148 layers. Near the best codes the estimate’s rank correlation falls to about 0.27. In the first run, the code with estimate 69 became a circuit of 167 layers, while the code with estimate 72 gave the 147. The spread comes from the last step of compilation, a scheduler that reorders gates which commute. Depth after the earlier compile steps ranks the final depth at only 0.35 to 0.36, and no cheap formula we tried predicted the scheduler.

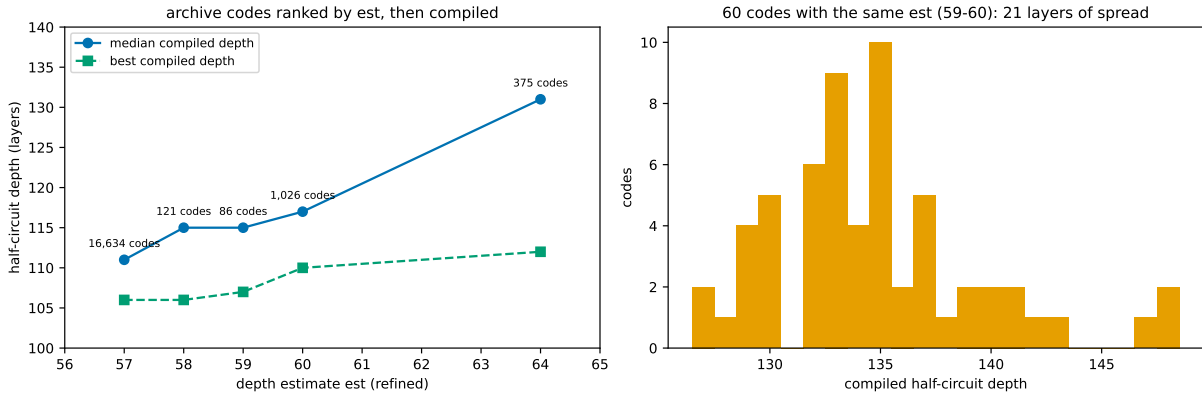


Figure 14: The estimate as a compass, not a judge. Left: archive codes grouped by estimate and compiled into half circuits; lower estimates give shallower circuits on average. Right: 60 codes with the same estimate (59–60) spread over 21 layers once compiled.

So the search ran as a funnel (figure 15). The annealer explores with the estimate, every correct code it prints is compiled, and only compiled depth decides what is kept and what seeds the next round. Pairing a fast, approximate search with a slow, exact check is also how recent quantum circuit optimisers work [16].

7 From 107 to 95: polishing a finished circuit

At 107 layers the code search stopped improving: every search we ran came back with the same best column code. The last twelve layers came from tools that change the circuit *without changing what it computes*. Each one either reorders gates that commute [17], or rebuilds a small block of the circuit with an exact solver [18].

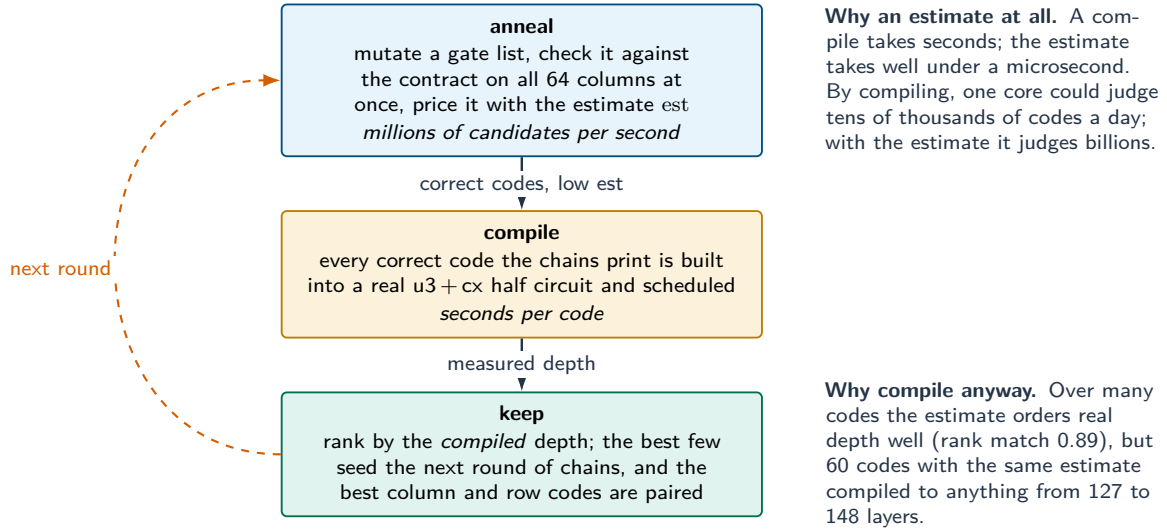


Figure 15: The search loop. The cheap estimate decides where to look; the compiler decides what to keep. The best codes of each round seed the next round, and the best column and row codes are paired.

depth / cx	what bought it
107/279	one gate that reads both sides, $\text{ccx}(q_0, \overline{q_{11}} \rightarrow q_3)$: six fewer cx
106/279	the phase block has 238 equivalent schedules; one of them saves a layer
98/269	a cheaper three-control gadget, the full <code>tket</code> [13] pass, and an exact CP-SAT [19] reschedule
97/267	how each multi-control gadget is compiled, chosen for pairs of neighbouring gadgets at once
96/273	one-qubit gates moved out of runs of gates they commute with
95/279	the phase block rebuilt by a SAT solver [20] inside the rest of the circuit
95/272	the same block with 19 cx, the fewest this block can have

Figure 16 puts these steps at the end of the whole history. The largest single drops came from changing what we searched over: one sandwich instead of three, then codes instead of circuits. Polishing paid steadily but in small steps.

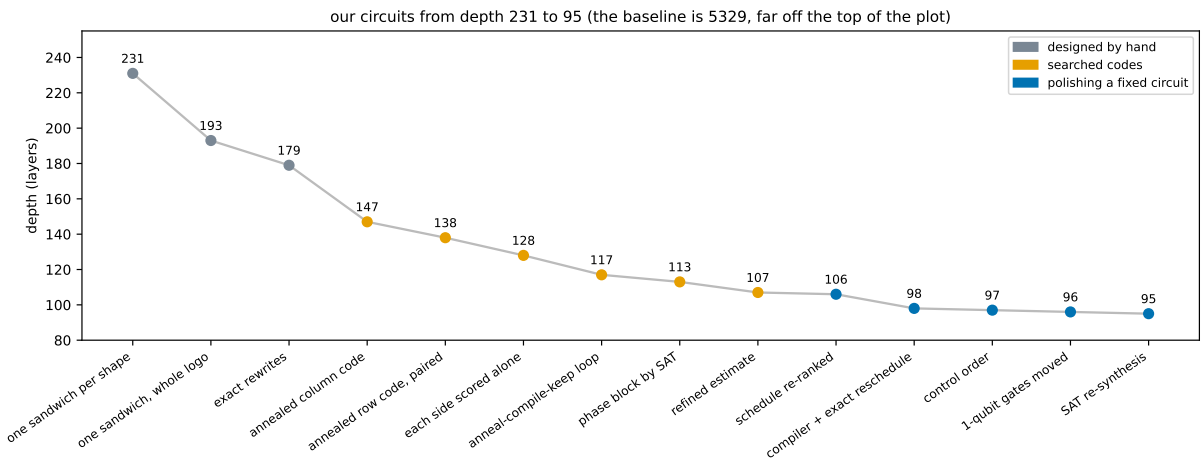


Figure 16: Our circuits from depth 231 to 95, each point a stored circuit re-measured by the companion notebook. Grey: designed by hand. Orange: codes found by search. Blue: a finished circuit packed more tightly. The baseline, 5329, is far off the top of the plot.

7.1 Why it stops at 95

Figure 6 already showed the culprit: the helper q_{16} is busy in every one of the nine middle layers. Every longest chain of gates in the circuit runs through the phase block on q_{16} . If that block takes K layers on q_{16} , a solver fits the rest of the circuit in exactly $86 + K$ layers, for every K from 0 to 9. And the block cannot take fewer than 9 layers on q_{16} : the solver proves 8 impossible even when the block's other qubits get all the time they want. So $86 + 9 = 95$. A circuit of depth 94 needs a different middle, one whose phase block spends fewer layers on q_{16} , or a comparison that hands q_{16} over sooner.

8 What did not work, and what we would keep

8.1 Routes that did not go below 106

Before polishing, we spent a long time trying to make the codes themselves shorter than those of the 106-layer circuit. Figure 17 sorts the routes we closed by why they failed. Most of them ran out of *helper qubits*: a clean spare qubit would have helped, and with six helpers neither side could give one up. Others made one side cheaper only by making the other side's contract tighter, and the column side, which sets the depth, always paid. A few searches spent their whole budget and returned nothing. The last group failed because a number measured in one setting was reused in another; re-measuring was itself worth a layer.

8.2 What carried the result

1. **Pick a shape that packs.** One sandwich with the two codes on disjoint qubits costs the longer code, where per-shape sandwiches cost the sum of all of them.
2. **Describe the picture with small codes.** A radius per column and a distance per row turn four shapes into one comparison and a few products of flags.
3. **Write down the freedom.** The contract lists every word each column and each row may use. It makes the two codes independent, so pairing costs nothing, and it makes checking a candidate a table lookup.
4. **Search with a cheap compass, decide with the real compiler.** The estimate steered billions of annealing steps; only compiled depth decided what we kept.
5. **Polish last.** Exact rescheduling and SAT re-synthesis of small blocks took the final twelve layers without touching the codes.

8.3 Tools we used

Besides our own annealer, the work relied on open tools. Code circuits were compiled with `tket` [13] and the Qiskit transpiler [14]. Exact rescheduling used the CP-SAT solver of Google OR-Tools [19]. The phase block was re-synthesised as a SAT problem, encoded with PySAT [21], solved with CaDiCaL [20] and Kissat [22]. The first whole-logo circuits were evolved with ShinkaEvolve [5].

8.4 Where to find everything

Everything below is in the folder that comes with this report.

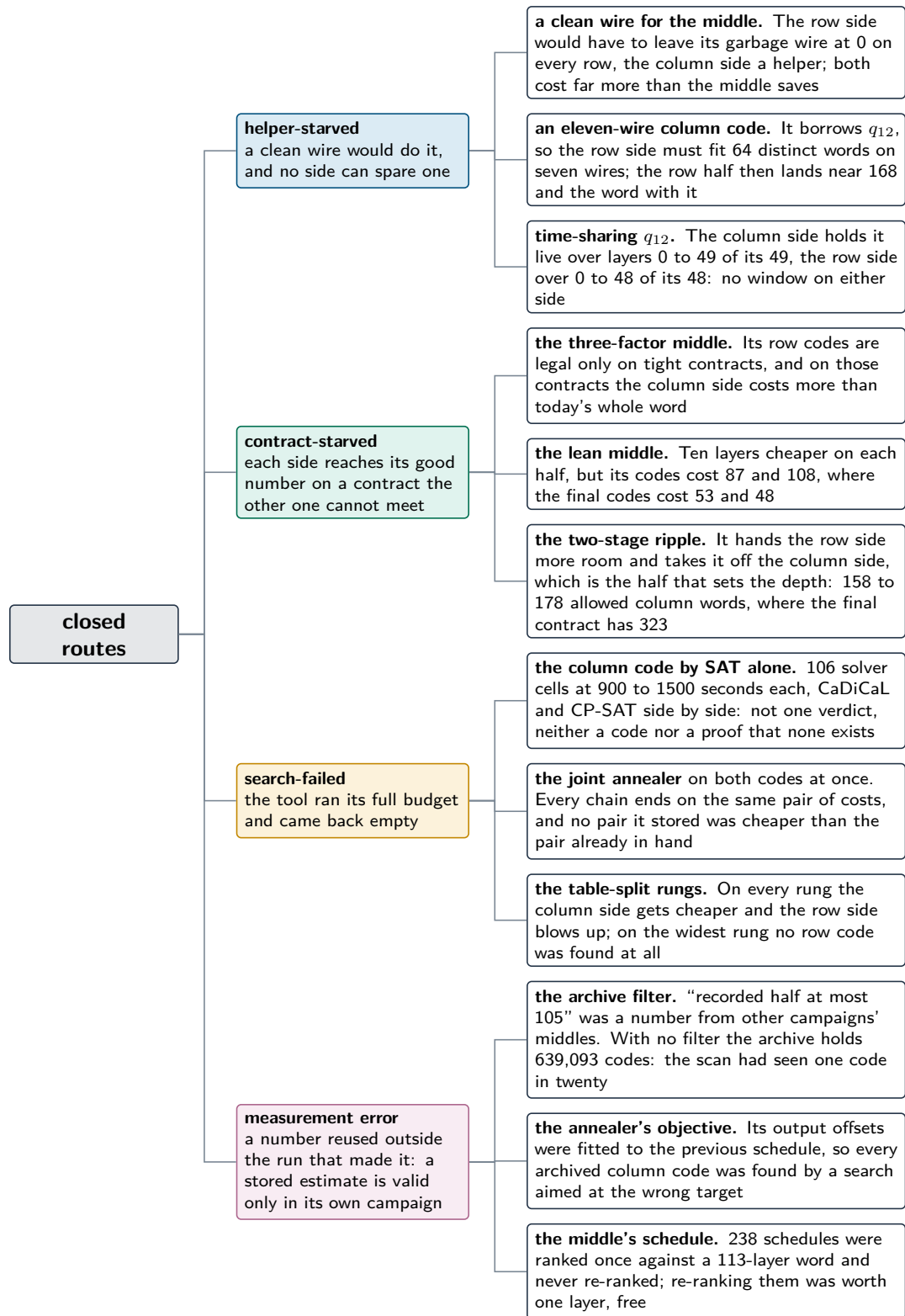


Figure 17: Closed routes, sorted by why they failed. Three of the four reasons are about resources (helper qubits, contract room, search budget), and the fourth is about stale measurements.

this report	logo-oracle-depth-95-explainer.pdf
companion notebook	classiq-challenge-depth-95-explainer.ipynb
its one helper script	depth95_helpers.py
the final circuit	data/oracle-depth-95-cx-272-6cdddada.qasm
every circuit of figure 16	data/ladder/
the annealer (C source)	data/colsa.c
codes, contracts, a pairing run	data/rec138.json, data/*-contract-138.json, data/pair1/
annealing logs, proxy test	data/run1/, data/col-60.jsonl

References

- [1] Classiq Technologies. *Classiq Coding Competition: Phase Oracle Depth Challenge (unitaryHACK 2026)*. Challenge page, leaderboard and reference notebook. 2026. URL: <https://www.classiq.io/challenge> (visited on 09/15/2026).
- [2] Lov K. Grover. “A Fast Quantum Mechanical Algorithm for Database Search”. In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC ’96)*. 1996, pp. 212–219. DOI: [10.1145/237814.237866](https://doi.org/10.1145/237814.237866).
- [3] Charles H. Bennett. “Logical Reversibility of Computation”. In: *IBM Journal of Research and Development* 17.6 (1973), pp. 525–532. DOI: [10.1147/rd.176.0525](https://doi.org/10.1147/rd.176.0525).
- [4] Dmitri Maslov. “Advantages of Using Relative-Phase Toffoli Gates with an Application to Multiple Control Toffoli Optimization”. In: *Physical Review A* 93.2 (2016), p. 022311. DOI: [10.1103/PhysRevA.93.022311](https://doi.org/10.1103/PhysRevA.93.022311).
- [5] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. *ShinkaEvolve: Towards Open-Ended and Sample-Efficient Program Evolution*. 2025. arXiv: [2509.19349](https://arxiv.org/abs/2509.19349) [cs.AI]. URL: <https://arxiv.org/abs/2509.19349>.
- [6] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. “Mathematical Discoveries from Program Search with Large Language Models”. In: *Nature* 625.7995 (2024), pp. 468–475. DOI: [10.1038/s41586-023-06924-6](https://doi.org/10.1038/s41586-023-06924-6).
- [7] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. *A New Quantum Ripple-Carry Addition Circuit*. 2004. arXiv: [quant-ph/0410184](https://arxiv.org/abs/quant-ph/0410184) [quant-ph]. URL: <https://arxiv.org/abs/quant-ph/0410184>.
- [8] Bruno Schmitt, Fereshte Mozafari, Giulia Meuli, Heinz Rienner, and Giovanni De Micheli. “From Boolean Functions to Quantum Circuits: A Scalable Quantum Compilation Flow in C++”. In: *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 2021, pp. 1044–1049. DOI: [10.23919/DATE51398.2021.9474237](https://doi.org/10.23919/DATE51398.2021.9474237).
- [9] Daniel Große, Robert Wille, Gerhard W. Dueck, and Rolf Drechsler. “Exact Synthesis of Elementary Quantum Gate Circuits for Reversible Functions with Don’t Cares”. In: *38th International Symposium on Multiple Valued Logic (ISMVL 2008)*. 2008, pp. 214–219. DOI: [10.1109/ISMVL.2008.42](https://doi.org/10.1109/ISMVL.2008.42).
- [10] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. “Optimization by Simulated Annealing”. In: *Science* 220.4598 (1983), pp. 671–680. DOI: [10.1126/science.220.4598.671](https://doi.org/10.1126/science.220.4598.671).
- [11] Eric Schkufza, Rahul Sharma, and Alex Aiken. “Stochastic Superoptimization”. In: *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’13)*. 2013, pp. 305–316. DOI: [10.1145/2451116.2451150](https://doi.org/10.1145/2451116.2451150).

- [12] Anouk Paradis, Jasper Dekoninck, Benjamin Bichsel, and Martin Vechev. “Synthetiq: Fast and Versatile Quantum Circuit Synthesis”. In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 55–82. DOI: [10.1145/3649813](https://doi.org/10.1145/3649813).
- [13] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. “t|ket>: A Retargetable Compiler for NISQ Devices”. In: *Quantum Science and Technology* 6.1 (2021), p. 014003. DOI: [10.1088/2058-9565/ab8e92](https://doi.org/10.1088/2058-9565/ab8e92).
- [14] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. *Quantum Computing with Qiskit*. 2024. arXiv: [2405.08810](https://arxiv.org/abs/2405.08810) [quant-ph]. URL: <https://arxiv.org/abs/2405.08810>.
- [15] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. “Elementary Gates for Quantum Computation”. In: *Physical Review A* 52.5 (1995), pp. 3457–3467. DOI: [10.1103/PhysRevA.52.3457](https://doi.org/10.1103/PhysRevA.52.3457).
- [16] Amanda Xu, Abtin Molavi, Swamit Tannu, and Aws Albarghouthi. “Optimizing Quantum Circuits, Fast and Slow”. In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS ’25)*. 2025, pp. 777–793. DOI: [10.1145/3669940.3707240](https://doi.org/10.1145/3669940.3707240).
- [17] Toshinari Itoko and Takashi Imamichi. “Scheduling of Operations in Quantum Compiler”. In: *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. arXiv:2011.04936. 2020, pp. 337–344. DOI: [10.1109/QCE49297.2020.00049](https://doi.org/10.1109/QCE49297.2020.00049).
- [18] Tom Peham, Nina Brandl, Richard Kueng, Robert Wille, and Lukas Burgholzer. “Depth-Optimal Synthesis of Clifford Circuits with SAT Solvers”. In: *IEEE International Conference on Quantum Computing and Engineering (QCE 2023)*. arXiv:2305.01674. 2023, pp. 802–813. DOI: [10.1109/QCE57702.2023.00095](https://doi.org/10.1109/QCE57702.2023.00095).
- [19] Laurent Perron and Frédéric Didier. *CP-SAT*. Google OR-Tools, v9.15. 2024. URL: https://developers.google.com/optimization/cp/cp_solver/.
- [20] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. “CaDiCaL 2.0”. In: *Computer Aided Verification – 36th International Conference, CAV 2024, Part I*. Vol. 14681. Lecture Notes in Computer Science. Springer, Cham, 2024, pp. 133–152. DOI: [10.1007/978-3-031-65627-9_7](https://doi.org/10.1007/978-3-031-65627-9_7).
- [21] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. “PySAT: A Python Toolkit for Prototyping with SAT Oracles”. In: *Theory and Applications of Satisfiability Testing – SAT 2018*. Vol. 10929. Lecture Notes in Computer Science. Springer, Cham, 2018, pp. 428–437. DOI: [10.1007/978-3-319-94144-8_26](https://doi.org/10.1007/978-3-319-94144-8_26).
- [22] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. “CaDiCaL, Gimsatul, IsaSAT and Kissat Entering the SAT Competition 2024”. In: *Proceedings of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions*. Ed. by Marijn Heule, Markus Iser, Matti Järvisalo, and Martin Suda. Vol. B-2024-1. Department of Computer Science Report Series B. University of Helsinki, 2024, pp. 8–10. URL: <https://cca.informatik.uni-freiburg.de/papers/BiereFallerFazekasFleuryFroleyksPollitt-SAT-Competition-2024-solvers.pdf>.